



**FATİH
SULTAN
MEHMET**
VAKIF ÜNİVERSİTESİ

Öğrenci:

Ad: Ali Enes

Soyad: GÜREL

Numara: 2121221032

Bölüm: Bilgisayar Mühendisliği

Proje:

Adı: Derin Öğrenmeye Giriş Bitirme Projesi Ödevi

Kullanılan Programlama Dili: Python

Proje Dili: Türkçe

Ders:

Adı: Introduction to Deep Learning

Eğitmen: Dr. Öğr. Üyesi Ümit DEMİRBAGA

Teslim Tarihi: 07.06.2026

İçindekiler

1. ÖZET	3
2. GİRİŞ	3
3. SEÇİLEN ÜÇ MODELİN AÇIKLANMASI.....	3
3.1. Convolutional Neural Networks (CNN) ile Hava Durumu Sınıflandırması	3
3.2. Autoencoder ile Görüntü Renklendirme (Image Colorization).....	4
3.3. Recurrent Neural Networks (RNN / LSTM) ile Nefret Söylemi Tespiti.....	4
4. VERİ SETLERİNİN AÇIKLANMASI VE VERİ SETİ LİNKLERİ	5
4.1. CNN Hava Durumu Sınıflandırma Veri Seti.....	5
4.2. Autoencoder (Görüntü Renklendirme) Veri Seti.....	6
4.3. RNN (LSTM) Nefret Söylemi ve Siber Zorbalık Veri Seti.....	6
5. MODEL MİMARİLERİ.....	7
5.1. Hava Durumu Sınıflandırması İçin CNN Mimarisi	7
5.2. Görüntü Renklendirme İçin Micro U-Net (Autoencoder) Mimarisi	8
5.3. Nefret Söylemi Tespiti İçin LSTM (RNN) Mimarisi.....	9
6. EĞİTİM SÜRECİ.....	10
6.1. CNN ile Hava Durumu Sınıflandırması Eğitim Süreci	10
6.2. Micro U-Net (Autoencoder) ile Görüntü Renklendirme Eğitim Süreci	12
6.3. LSTM (RNN) ile Nefret Söylemi Tespiti Eğitim Süreci.....	14
7. DENEYSEL SONUÇLAR.....	16
7.1. CNN Hava Durumu Sınıflandırma Performansı	16
7.2. Micro U-Net Autoencoder Performans Analizi.....	16
7.3. RNN (LSTM) Siber Zorbalık Tespit Başarısı	16
8. GRAFİKLER VE YORUMLAR	17
8.1. Hava Durumu Sınıflandırması (CNN).....	17
8.1.1 Grafik ve Yorumları	17
8.1.2 Prediction Örnekleri	18
8.2. Görüntü Renklendirme (Micro U-Net Autoencoder)	19
8.2.1 Grafik ve Yorumları	19
8.2.2 Prediction Örnekleri	20
8.3. Metin Analizi ile Nefret Söylemi Tespiti (LSTM).....	21
8.3.1 Grafik ve Yorumları	21
8.3.2 Prediction Örnekleri	22
8.4. Kodların Çalıştırılması ve Kullanılan Kütüphaneler	23
9. TARTIŞMA.....	23
10. SONUÇ	24
11. KAYNAKÇA	25

1. ÖZET

Bu projede, derin öğrenmede kullanılan yapay sinir ağı mimarilerinden üç tanesi olan Convolutional Neural Networks and Applications (CNN), Autoencoder ve Recurrent Neural Networks (RNN) seçilerek farklı veri setleri üzerinde uygulandı. Projedeki istenilenler doğrultusunda iki görüntü (image) ve bir metin (text) veri seti işlendi.

İlk uygulamada, "Hava Durumu Sınıflandırması" için CNN modeli kullanıldı ve %84.44 doğruluk oranına ulaşıldı. İkinci uygulamada, siyah-beyaz görselleri renklendirmek amacıyla "skip connection" yapısına sahip Micro U-Net Autoencoder kullanıldı, doğrulama kaybı (MSE) 0.0062 seviyesine indirilerek model eğitimindeki piksel doğruluğunda %53.26, ikili aydınlık/karanlık piksel sınıflandırmasında ise %93 genel başarı oranına ulaşıldı. Üçüncü uygulamada ise X (Twitter) platformundaki "Nefret Söylemi ve Siber Zorbalık Tespiti" için RNN (LSTM) ağı tasarlandı, model metin sınıflandırmasında %75.49 genel doğruluk elde edildi.

Modelleri eğitirken overfitting'in önüne geçmek için erken durdurma yani EarlyStopping yöntemi kullanıldı. Böylece eğitim süreci kontrol altında tutuldu. Elde edilen sonuçlar ise accuracy, loss ve güven skorları üzerinden akademik olarak analiz edildi.

2. GİRİŞ

Geleneksel makine öğrenmesi yöntemleri, görüntü ve metin gibi yapılandırılmamış verilerin analizinde çoğunlukla manuel öznitelik çıkarımına ihtiyaç duyarken, derin öğrenme yaklaşımları verideki hiyerarşik temsilleri yapay sinir ağları aracılığıyla uçtan uca öğrenerek bu ihtiyacı büyük ölçüde azaltmaktadır.

Bu projede, farklı veri türlerine yönelik üç derin öğrenme mimarisinin pratik uygulamalardaki performansı incelendi. Bu doğrultuda CNN modeli, görüntülerdeki uzamsal özellikleri öğrenme kapasitesi nedeniyle çok sınıflı hava durumu sınıflandırma probleminde kullanıldı. Autoencoder Micro U-Net mimarisi, skip connection yapıları ile desteklenerek görüntü renklendirmede piksel tabanlı restorasyon amacıyla uygulandı. RNN(LSTM) modeli ise ardışık veri yapılarında bağlam bilgisini yakalama yeteneği sayesinde X (Twitter) platformundan elde edilen metinlerde nefret söylemi ve siber zorbalık tespiti için tercih edildi.

3. SEÇİLEN ÜÇ MODELİN AÇIKLANMASI

3.1. Convolutional Neural Networks (CNN) ile Hava Durumu Sınıflandırması

- Uygulamanın Amacı: Gökyüzü ve atmosfer fotoğraflarını inceleyerek havanın durumunu tahmin etmek.
- Kullanılan Veri Türü: Görüntü (Image).
- Problemin Türü: Çok sınıflı sınıflandırma (Multi-class classification).
- Modelin Uygunluğu: CNN mimarisi, görüntü üzerinde dolaşan filtreleri sayesinde bulut yoğunluğu, yağmur veya ışık kırılmaları gibi karmaşık spatial detayları bir bütün olarak algılamada üstün başarı gösterdiği için bu problemde kullanıldı.

Görsel verilerin analizinde ANN diye adlandırılan geleneksel yapay sinir ağlarının en büyük sorunu, görüntüleri tek boyutlu bir vektöre indirgemeleri ve pikseller arasındaki konumsal yani spatial ilişkileri kaybetmeleridir. Bu noktada pikselleri tek tek değil, belirli komşuluk ilişkileriyle bir bütün olarak tarayan Evrişimli Sinir Ağları (CNN) devreye girer.

CNN mimarisinin temelinde, görüntü üzerinde dolaşan kernellar sayesinde kenar, köşe, doku ve renk geçişleri gibi önemli detayların algılanması yer alır. Convolution katmanları bu özellikleri öğrenir, pooling katmanları ise görüntünün boyutunu küçülterek gereksiz bilgileri azaltır. Böylece hem işlem maliyeti düşer hem de model, bir nesnenin görüntüde tam olarak nerede bulunduğundan bağımsız olarak onu tanımayı öğrenir.

Projenin ilk uygulamasında CNN mimarisi, gökyüzü ve atmosfer fotoğraflarını inceleyerek havanın durumunu tahmin etmek amacıyla kullanıldı. Hava durumu sınıflandırması, bir görselin içindeki bulut yoğunluğu, ışık kırılmaları veya yağmur damlası çizgileri gibi karmaşık dokuların ayırt edilmesini gerektirir. Model, katmanları derinleştikçe "shine", "cloudy", "sunrise" ve "rain" sınıflarına ait piksel bazlı örüntüleri kademeli olarak öğrendi. Böylece yüksek doğruluk oranına ulaşıldı.

3.2. Autoencoder ile Görüntü Renklendirme (Image Colorization)

- Uygulamanın Amacı: Siyah-beyaz formattaki görüntüleri orijinal bağlamına ve nesne yapılarına uygun şekilde renklendirmek.
- Kullanılan Veri Türü: Görüntü (Image).
- Problemin Türü: Görüntü restorasyonu ve piksel regresyonu.
- Modelin Uygunluğu: Klasik autoencoder yapılarında darboğaz (bottleneck) kısmında ince detaylar kaybolurken, Micro U-Net yapısındaki "skip connections" yüksek çözünürlüklü özelliklerin korunmasını sağlayarak gerçekçi bir renklendirme (image colorization) işlemi sunar.

Autoencoder yapıları, girdiyi doğrudan bir etikete dönüştürmeden veriyi anlamlı bir şekilde yeniden üretmeyi amaçlayan denetimsiz öğrenme modelleridir. Temelde Kodlayıcı (Encoder) ve Kod Çözücü (Decoder) olmak üzere iki bölümden oluşur. Encoder, yüksek boyutlu veriyi daha küçük ve anlamlı bir temsil alanına sıkıştırırken, gereksiz detayları ve gürültüleri ayıklayarak verinin en önemli özelliklerini korumaya çalışır. Decoder ise bu sıkıştırılmış koddan yola çıkarak veriyi yeniden oluşturmaya çalışır.

Projenin ikinci uygulamasında bu mimari, siyah-beyaz görüntüleri renklendirmek amacıyla kullanıldı. Görüntü renklendirme problemi, yalnızca eksik renkleri doldurmaktan ibaret değildir; aynı zamanda nesnelerin yapısını, ışık geçişlerini ve sahnenin genel bağlamını doğru yorumlamayı gerektirir. Modelin görevi, gri tonlamalı bir görüntüde hangi pikselin hangi RGB renk değerine karşılık geldiğini öğrenmektir.

Ancak klasik autoencoder yapılarında model derinleştikçe görüntüye ait ince detaylar ve keskin kenarlar kaybolabilir. Bu durum, çıktının bulanık görünmesine neden olur. Bu problemi azaltmak için projede, U-Net mimarisinin daha hafif bir versiyonu olan Micro U-Net tercih edilmiştir. Bu yapının en önemli avantajı, encoder katmanlarında öğrenilen detaylı özelliklerin "skip connection" adı verilen bağlantılar aracılığıyla doğrudan decoder katmanlarına aktarılmasıdır. Böylece model, görüntünün geometrik yapısını koruyarak daha net, tutarlı ve gerçekçi renklendirme sonuçları üretebilmektedir.

3.3. Recurrent Neural Networks (RNN / LSTM) ile Nefret Söylemi Tespiti

- Uygulamanın Amacı: X (Twitter) platformundan elde edilen metinleri analiz ederek paylaşımları normal veya siber zorbalık/nefret söylemi olarak sınıflandırmak.
- Kullanılan Veri Türü: Metin (Text).
- Problemin Türü: İkili sınıflandırma (Binary classification).
- Modelin Uygunluğu: Sosyal medya dilinde kelimelerin sırası ve bağlamı cümlelerin niyetini tamamen değiştirebildiğinden, tekrarlayan ve geçmişini hafızasında tutabilen LSTM mimarisi bu ardışık veri problemi için en uygun seçenektir.

Görüntü verilerinden farklı olarak metin, ses veya zaman serisi gibi veriler sequential bir yapıya sahiptir. Bu tür verilerde anlam, yalnızca içerikten etkilenmez, bilgilerin hangi sırayla geldiğinden de doğrudan etkilenir. Geleneksel ileri beslemeli ağlar ve CNN mimarileri veriyi tek seferde işler, bu nedenle önceki adımlardaki bilgileri hafızalarında tutamazlar. Tekrarlayan Sinir Ağları (RNN) ise döngüsel yapıları sayesinde bir adımdan elde edilen bilgiyi sonraki adıma aktararak geçmişini koruyabilen bir yapı sunar.

Ancak klasik RNN modelleri, özellikle uzun metinlerde zamanla eski bilgileri unutmaya başlarlar. Bu önemli bir sorundur ve "vanishing gradient" problemi olarak bilinir. Bu da modelin cümlelerin başındaki bilgileri sonuna kadar taşımasını zorlaştırır. Bu sorunu azaltmak için geliştirilen LSTM (Long Short-

Term Memory) mimarisi, özel kapı mekanizmaları sayesinde hangi bilginin tutulacağına, hangisinin unutulacağına ve hangisinin aktarılacağına karar vererek daha güçlü bir hafıza yapısı oluşturur.

Projenin üçüncü uygulamasında, Twitter (X) platformundan elde edilen metin verileri kullanılarak siber zorbalık, ofansif dil ve nefret söylemi tespiti gerçekleştirildi. Sosyal medya dili çoğu zaman düzensiz, kısa ve bağlama oldukça bağımlıdır. Bir kelime tek başına masum görünse bile, birlikte kullanıldığı diğer ifadelerle farklı ve saldırgan bir anlam kazanabilir. Bu nedenle yalnızca kelimeleri tek tek incelemek yeterli olmaz; cümlelerin genel akışını ve bağlamını anlamak gerekir.

Kurulan LSTM tabanlı model, metindeki kelimeler arasındaki anlamsal ilişkileri ve sıralı yapıyı takip ederek paylaşımları “Siber Zorbalık” ve “Normal” olarak sınıflandırmayı öğrendi. Böylece model, yalnızca belirli anahtar kelimelere değil, ifadelerin taşıdığı genel niyete ve bağlama göre de değerlendirme yapabilmektedir.

4. VERİ SETLERİNİN AÇIKLANMASI VE VERİ SETİ LİNKLERİ

4.1. CNN Hava Durumu Sınıflandırma Veri Seti

Veri Seti Adı: Weather Dataset for Image Classification

Veri Seti Linki: <https://www.kaggle.com/datasets/orvile/weather-dataset-for-image-classification>

Veri Seti Türü: Görüntü (Image)

Veri Setinin Uygunluğu: Bu veri seti, hava durumunu belirleyen bulut dokuları, ışık yansımaları ve yağmur çizgileri gibi uzamsal özelliklerin (spatial features) net bir şekilde ayırt edilebildiği gerçek dünya fotoğrafları içerdiğinden, CNN mimarisinin filtreleme yeteneğini test etmek için son derece uygundur.

Veri Ön İşleme Adımları:

Atmosfer ve gökyüzü olaylarının çok sınıflı sınıflandırması (cloudy, rain, shine, sunrise) için Kaggle'dan elde edilen toplam 1.126 görsellik bir veri seti kullanıldı. Ham veri seti indirildiğinde, tüm fotoğrafların tek bir klasör içerisinde etiketsiz ve karmaşık bir yapıda bulunduğu için bu veriyi eğitime hazır hale getirmek için bazı ön işleme adımları uygulandı.

- İşletim Sistemi Seviyesinde Otomatik Ayrıştırma:** Karmaşık haldeki görselleri sınıflandırmak için Python'un os ve shutil kütüphaneleriyle özel bir dosya okuma algoritması geliştirildi. Bu algoritma, dosya isimlerinin başlangıcındaki (prefix) metinleri taradı ve görüntüleri dataset_Duzenli ana dizini altında oluşturulan 4 ayrı etiket klasörü olan cloudy, rain, shine, sunrise'a otomatik olarak kopyalayarak veriyi yapılandırdı.
- Eğitim ve Doğrulama (Validation) Setlerinin Ayrılması:** Klasörlenen görseller, tf.keras.preprocessing.image_dataset_from_directory metodu kullanılarak yüklendi. Modelin veriyi ezberlemesini (overfitting) önlemek ve genelleme performansını güvenilir şekilde ölçmek amacıyla veri seti; rastgelelik ilkesine (seed=123) sadık kalınarak %80 eğitim (901 görsel) ve %20 doğrulama (225 görsel) oranında ikiye bölündü.
- Yeniden Boyutlandırma ve Normalizasyon:** Hesaplama maliyetini düşürmek ve modelin gerektirdiği sabit matris girdisini sağlamak için tüm görseller 128x128 piksel boyutuna getirildi. Ayrıca, ağır gradyan güncellemelerini hızlandırmak adına, modelin Input aşamasına entegre edilen Rescaling katmanı ile piksel değerleri 255'e bölünerek [0, 1] aralığına normalize edildi.
- I/O Optimizasyonu ve Veri Boru Hattı (Pipeline):** Model eğitimi sırasındaki bellek darboğazlarını ve GPU bekleme sürelerini ortadan kaldırmak için yığın boyutu batch_size=32 olarak belirlendi. Eğitim verileri belleğe alınmış (.cache()), ezberlemeyi önlemek için karıştırılmış (.shuffle(1000)) ve TensorFlow'un AUTOTUNE destekli .prefetch() mekanizması devreye sokularak donanım kaynaklarının eşzamanlı kullanımı maksimize edildi.

4.2. Autoencoder (Görüntü Renklendirme) Veri Seti

Veri Seti Adı: Image Colorization

Veri Seti Linki: <https://www.kaggle.com/datasets/seungjunleeofficial/image-colorization>

Veri Seti Türü: Görüntü (Image)

Veri Setinin Uygunluğu: Colorization yani görüntü renklendirme problemi, piksellerin bağlamsal olarak onarılmasını gerektirir. Bu veri seti, mimarideki "skip connection" yapısının yüksek çözünürlüklü kenar detaylarını nasıl koruduğunu ölçmek için ideal olan, geniş bir nesne ve manzara çeşitliliğine sahiptir.

Veri Ön İşleme Adımları:

Görüntü renklendirme (Colorization) problemi için Kaggle platformunda bulunan "Image Colorization" veri seti kullanıldı. Veri seti indirildiğinde eğitim (train) ve doğrulama (val) verilerinin halihazırda ayrılmıştı ve bu klasörlerin altında siyah-beyaz girdilerin gray, orijinal renkli hedeflerin ise color klasörleri içerisinde konumlandığı görüldü.

Modelin siyah-beyaz bir girdi ile ona ait olmayan yanlış bir renkli çıktıyı eşleştirmesini engellemek son derece kritiktir. Bu nedenle, Python'un işletim sistemi araçları (OS) kullanılarak get_matched_paths isimli özel bir eşleştirme algoritması geliştirildi. Bu algoritma; iki klasördeki .png, .jpg ve .jpeg uzantılı dosyaları ismen tarayarak birebir eşleştirmiş ve eğitim/doğrulama için ayrılmış kesin dosya yollarını array formatında toplamıştır.

Veri setinin hacmi nedeniyle bellek şişmelerini önlemek ve I/O darboğazlarını aşarak yüksek performanslı bir veri okuma süreci sağlamak amacıyla, tf.data.Dataset.from_tensor_slices metodu kullanılarak baştan uca bir pipeline inşa edildi.

Bu boru hattındaki ön işleme aşamasında, eşleşen her bir dosya yolu process_path isimli bir mapping fonksiyonundan geçirildi.

- TensorFlow'un io.read_file ve decode_jpeg metotları kullanılarak siyah-beyaz (girdi) veriler 1 kanallı (sadece luma/parlaklık değeri), renkli (hedef) veriler ise 3 kanallı (RGB) matrisler olarak okundu.
- Hesaplama maliyeti ile görüntü kalitesi arasındaki optimum dengeyi kurmak için tüm görseller 128x128 piksel boyutuna yeniden boyutlandırıldı.
- Modelin gradyan güncellemelerini hızlandırmak ve daha kararlı öğrenmesini sağlamak için piksel değerleri 255'e bölünerek [0, 1] aralığına normalize edildi.

Son olarak, ağır öğrenme sürecindeki I/O gecikmelerini kırmak için batch_size=32 olarak ayarlanmış ve donanım kaynaklarının kullanımını maksimize eden tf.data.AUTOTUNE destekli çoklu işlem (num_parallel_calls) ve .prefetch() mekanizmaları devreye alınmıştır.

4.3. RNN (LSTM) Nefret Söylemi ve Siber Zorbalık Veri Seti

Veri Seti Adı: Hate speech and Offensive language dataset from X

Veri Seti Linki: <https://www.kaggle.com/datasets/nahidhasannill/hate-speech-and-offensive-language-dataset-from-x>

Veri Seti Türü: Metin (Text)

Veri Setinin Uygunluğu: X (Twitter) platformundan alınan bu veri, günlük sosyal medya dilinin içerdiği düzensiz gramer, ironi ve örtülü nefret söylemlerini barındırmaktadır. Bu durum, LSTM modelinin kelimeler arasındaki anlam ve zaman ilişkilerini öğrenme becerisini test etmek için uygun bir senaryo sağlar.

Veri Ön İşleme Adımları:

Doğal dil işleme (NLP) uygulaması için, metin verilerinin yapısal olarak en kurlsız olduđu X (Twitter) platformundan çekilmiş Kaggle veri seti kullanıldı. Ham CSV dosyasında yer alan veriler doğrudan matematiksel bir modele verilemeyeceğı için, aşağıdaki adımları içeren kapsamlı bir ön işleme zinciri kuruldu:

- **Etiketlerin İkili (Binary) Formata Dönüştürülmesi:** Sınıflandırma işlemini optimize etmek adına veri setindeki çoklu ofansif dil kategorileri sadeleştirildi. "Nefret" (1) ve "Saldırgan" (2) etiketleri birleştirilerek "Siber Zorbalık/Ofansif" (1) sınıfına; "Normal" (3) etiketleri ise "Zorbalık Yok" (0) sınıfına maplenerek problem ikili sınıflandırma formatına getirildi.
- **Eksik Verilerin Temizlenmesi (Dropna):** Eğitim sürecinde hatalı matris işlemlerinin önüne geçmek için veri setinde tweet metninin veya hedeflenen hedeflenen etiketin boş (NaN) olduđu satırlar tespit edilerek veri setinden tamamen silindi.
- **Metinlerin Sayısallaştırılması (Tokenization):** Keras'ın Tokenizer fonksiyonu kullanılarak, veri setinde en sık geçen 10.000 kelimelik bir sözlük çıkarıldı. Ardından metin tabanlı olan tüm tweetler, bu sözlükteki ağırlıklarına göre sayısal dizilere dönüştürüldü.
- **Matris Boyutlarının Eşitlenmesi (Padding):** Her tweetin kelime uzunluđu birbirinden farklı olduđu için, LSTM ağıının matris çarpımlarında gerektirdiğı sabit girdi boyutunu sağlamak amacıyla maksimum kelime sınırı 100 olarak belirlendi. Keras'ın pad_sequences fonksiyonu ile 100 kelimedenden kısa olan tweetlerin eksik kısımlarına sıfırlar eklenerek diziler standartlaştırıldı.
- **Eğitim ve Test Kümelerinin Ayrılması:** Sayısallaştırma ve boyut eşitleme işlemleri tamamlanan veri seti, modelin genelleme yeteneğini ölçebilmek için rastgelelik ilkesine (random_state=42) sadık kalınarak %80 train ve %20 test verisi olacak şekilde ikiye ayrıldı.

5. MODEL MİMARİLERİ

Bu projede kullanılan üç farklı derin öğrenme modeli de TensorFlow ve Keras kütüphaneleri kullanılarak en baştan tasarlandı.

5.1. Hava Durumu Sınıflandırması İçin CNN Mimarisi

```
num_classes = len(class_names)

model = models.Sequential([
    # Piksel değerlerini [0, 255] aralığından [0, 1] aralığına normalize eder
    layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),

    # Görüntüden kenar ve şekil gibi uzamsal özellikleri çıkaran Evrişim (Conv2D) katmanları
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(), # Özellik matrisinin boyutunu küçülterek hesaplama yükünü azaltır
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),

    # Çok boyutlu matrisi tek boyutlu vektöre düzleştirir
    layers.Flatten(),
    layers.Dense(128, activation='relu'),

    # Ağırlıkların rastgele %50'sini kapatarak modelin veriyi ezberlemesini (Overfitting) engeller
    layers.Dropout(0.5), # OVERFITTING'İ ÖNLEMEK İÇİN
    # 4 sınıf için olasılık dağılımı üreten çıktı katmanı (softmax)
    layers.Dense(num_classes, activation='softmax')
])
```

Şekil 1 CNN modeli katman mimarisinin ve filtreleme adımlarının Keras ile oluşturulması

Görüntülerdeki karmaşık hava durumu örüntülerini (örneğin yağmur damlalarının çizgisel yapısı veya güneşin dairesel parlaması) yakalayabilmek için ardışık (Sequential) bir Evrişimli Sinir Ağı (CNN) kuruldu.

- **Conv2D Katmanları:** Görüntü üzerindeki kenar, renk ve şekil gibi özellikleri yakalamak için sırasıyla 32, 64 ve 128 kernel içeren evrişim katmanları kullanıldı. Negatif değerleri sönümlemek ve ağı non-linear bir öğrenme yeteneği kazandırmak için activation='relu' fonksiyonu tercih edildi.
- **MaxPooling2D:** Her evrişim adımından sonra, 2x2 matrislerle boyut küçültme işlemi yapıldı, böylece hem hesaplama maliyeti düşürüldü hem de modelin ezberlemesi engellendi.
- **Flatten ve Dense Katmanları:** Görüntüden elde edilen 2 boyutlu özellik haritaları Flatten() ile tek boyutlu vektöre dönüştürüldü. Çıktı katmanında 4 farklı hava durumu sınıfı olduğu için 4 nöronlu bir Dense katmanı ve olasılık dağılımı üreten softmax aktivasyon fonksiyonu kullanıldı.

Modelin Hiperparametreleri ve Gerekçelendirilmesi:

- **Optimizer ve Loss:** Adam optimizatörü ve çok sınıflı yapı nedeniyle categorical_crossentropy kullanıldı.
- **Batch Size ve Epoch:** Batch size 32 olarak ayarlandı, 15 epoch planlanan eğitim 11. epoch'ta erken durdurulmuştur.
- **Eğitim ve Test Ayrımı:** %80 (901 görsel) eğitim, %20 (225 görsel) doğrulama olarak ayrılmıştır.
- **Teknik Gerekçelendirme:** Hava durumu tahmini; bulutların şekli veya güneşin açısı gibi görüntü içindeki uzamsal hiyerarşilerin algılanmasını gerektirir. Klasik sinir ağları görüntüyü düzleştirip aldığı için mekansal komşuluk ilişkilerini kaybederken, CNN evrişim filtreleri sayesinde bu özellikleri bir bütün olarak yakalayabildiği için teknik olarak en uygun modeldir.

5.2. Görüntü Renklendirme İçin Micro U-Net (Autoencoder) Mimarisi

```
print("\n2. Mikro U-Net Kuruluyor...")
def build_micro_unet(input_shape=(img_size, img_size, 1)):
    inputs = layers.Input(shape=input_shape)

    # ENCODER (Kodlayıcı): Görüntünün boyutunu küçültüp, özellik haritalarını (feature maps) derinleştirir.
    c1 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(inputs)
    p1 = layers.MaxPooling2D((2, 2))(c1)
    c2 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(p1)
    p2 = layers.MaxPooling2D((2, 2))(c2)

    # BOTTLE-NECK (Darboğaz): Verinin en sıkıştırılmış (Latent space) halidir.
    bn = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(p2)

    # DECODER (Çözücü): Sıkıştırılmış veriyi tekrar orijinal boyutuna (128x128) yükseltir.
    # U-NET MİMARİSİNİN ÖZÜ (SKIP CONNECTIONS): Concatenate ile Encoder'daki yüksek çözünürlüklü detaylar
    # doğrudan Decoder'a aktarılır. Bu sayede kenar ve sınır çizgilerindeki bulanıklık önlenir.
    u1 = layers.UpSampling2D((2, 2))(bn)
    concat1 = layers.concatenate([u1, c2])
    c3 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(concat1)

    u2 = layers.UpSampling2D((2, 2))(c3)
    concat2 = layers.concatenate([u2, c1])
    c4 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(concat2)

    # ÇIKTI KATMANI: RGB (3 kanal) renkli görüntüyü oluşturur.
    # Sigmoid fonksiyonu piksel değerlerinin tam [0, 1] aralığına oturmasını garanti eder.
    outputs = layers.Conv2D(3, (3, 3), activation='sigmoid', padding='same')(c4)
    return models.Model(inputs, outputs)
```

Şekil 2 Micro U-Net Autoencoder mimarisinde atlama bağlantılarının (skip connections) kodlanması

Siyah-beyaz bir görüntüyü renklendirirken standart bir ardışık Autoencoder kullanmak, darboğaz kısmında yüksek frekanslı ince detayların kaybolmasına ve çıktının bulanık olmasına yol açar. Bu donanımsal kayıpları engellemek için model, "atlama bağlantılarına" yani Skip Connections'a sahip özel bir Micro U-Net formatında tasarlandı.

- **Encoder (Kodlayıcı) Aşaması:** 128x128 boyutlarındaki tek kanallı girdi, 16 ve 32 filtrelili Conv2D ve havuzlama katmanlarıyla sıkıştırılarak darboğaza doğru iletilmiştir.
- **Decoder (Çözücü) ve Concatenate (Birleştirme):** U-Net'i standart autoencoder'lardan ayıran kritik kod satırları buradadır. UpSampling2D ile matris boyutları büyütülürken, Concatenate() fonksiyonu sayesinde Encoder aşamasındaki yüksek çözünürlüklü özellik haritaları doğrudan Decoder aşamasına kopyalanmıştır. Bu sayede renklendirme sırasında nesnelerin kenar detayları kaybolmadı.
- **Çıktı Katmanı:** RGB görüntü üretimi hedeflendiği için son katmanda piksel değerlerini [0, 1] aralığına oturtmak amacıyla sigmoid aktivasyonu tercih edildi.

Modelin Hiperparametreleri ve Gerekçelendirilmesi:

- **Optimizer ve Loss:** Adam optimizatörü ve karesel piksel hatasını minimize etmek için MSE (Mean Squared Error) kullanıldı.
- **Batch Size ve Epoch:** I/O darboğazı yaratmamak için Batch size 32 olarak ayarlandı ve eğitimde erken durdurma olmadan planlanan 10 epoch tamamlandı.
- **Eğitim ve Test Ayrımı:** Veri setinin klasör hiyerarşisine sadık kalınarak 118.287 görsel train seti ve 5.000 görsel test seti olarak veri pipeline'ına doğrudan entegre edildi.
- **Teknik Gerekçelendirme:** Standart Autoencoder mimarilerinde veri sıkıştırıldığında nesnelerin mekansal detayları kaybolur. Görüntü renklendirmede kenar detaylarının çok iyi korunması gerektiğinden, Encoder'daki özellikleri Decoder'a taşıyan bu özel yapı tercih edildi.

5.3. Nefret Söylemi Tespiti İçin LSTM (RNN) Mimarisi

```
print("\n3. Metin Analizine Uygun LSTM Mimarisi Kuruluyor...")
model = Sequential([
    # Embedding: Kelimelerin birbiriyle olan anlamsal ilişkisini 64 boyutlu yoğun vektör uzayında temsil eder
    Embedding(max_features, 64, input_length=maxlen),

    # SpatialDropout1D: Standart dropout'tan farklı olarak, kelimeler arasındaki yapısal bağı koparmamak için
    # 1 boyutlu özellik haritalarını rastgele tümünden kapatarak ezberlemeyi engeller
    SpatialDropout1D(0.3),

    # LSTM Katmanı: Önceki kelimeleri hafızada tutarak cümlelerin genel niyetini/bağlamını (context) öğrenir
    LSTM(32, return_sequences=False),
    Dropout(0.5), # Ağırlıkların yarısını kapatarak genelleme yeteneğini artırır
    # Çıktı Katmanı: İkili sınıflandırma (0 veya 1) olduğu için Sigmoid aktivasyonu kullanılır
    Dense(1, activation='sigmoid')
])
```

Şekil 3 Ardışık metin verilerinin analizi için oluşturulan LSTM ağı ve hafıza birimleri

Sosyal medya metinlerindeki (Twitter/X) siber zorbalığı yakalamak, kelimelerin birbirleriyle olan zamansal ve anlamsal bağını (context) izlemeyi gerektirdiği için sıralı bellek yönetimli bir RNN (LSTM) mimarisi kuruldu.

- **Embedding Katmanı:** 10.000 kelimelik kelime dağarcığı, kelimeler arası anlamsal yakınlığı ifade edebilmek adına 64 boyutlu vektör uzayına (output_dim=64) dönüştürüldü.
- **SpatialDropout1D (0.3):** Standart dropout'tan farklı olarak, metin verilerindeki bağımlılıkları kırmamak adına 1 boyutlu feature maps tamamını %30 oranında rastgele kapatarak ağı ezberlemesinin önüne geçildi.

- **LSTM Katmanı:** 32 units'den oluşan bu katman, metin içindeki geçmiş kelimeleri hafızasında tutarak cümlelerin genel niyetini öğrendi.
- **Çıktı Katmanı:** İkili sınıflandırma yapılacağından ötürü tek bir nöron ve sigmoid fonksiyonu kullanıldı.

Modelin Hiperparametreleri ve Gerekçelendirilmesi:

- **Optimizer ve Loss:** Adam optimizatörü ve ikili sınıflandırma olduğu için binary_crossentropy kullanıldı.
- **Batch Size ve Epoch:** Batch size 128 olarak ayarlandı, 6 epoch planlanan eğitim 4. epoch'ta erken durdurulmuştur.
- **Eğitim ve Test Ayrımı:** %80 eğitim ve %20 test verisi olacak şekilde ikiye bölündü.
- **Teknik Gerekçelendirme:** Geleneksel Çok Katmanlı Algılayıcılar (MLP) kelime sırasını dikkate almaz. Sosyal medyadaki ironi ve nefret söylemini tespit edebilmek için modelin önceki kelimeleri hatırlaması gerekir. Verinin unutulması veya hatırlanması gereken kısımlarını gate mekanizmalarıyla kontrol eden LSTM, bu metin problemi için teknik olarak en rasyonel mimaridir.

6. EĞİTİM SÜRECİ

6.1. CNN ile Hava Durumu Sınıflandırması Eğitim Süreci

Veri setinin nasıl hazırlandığı:

```
# =====
# 1. KLASÖR DÜZENLEME (Otomatik Ayırıştırma)
# =====
# Karmaşık haldeki veri setini okuyup, dosya isimlerindeki etiketlere göre
# sınıflara ait alt klasörlere (cloudy, rain, vb.) otomatik olarak dağıtır.
source_dir = r'C:\Users\alien\archive\dataset2\dataset2'
target_base_dir = r'C:\Users\alien\archive\dataset_Duzenli'
classes = ['cloudy', 'rain', 'shine', 'sunrise']

if not os.path.exists(target_base_dir):
    os.makedirs(target_base_dir, exist_ok=True)
    print("Resimler sınıflarına göre ayrıştırılıyor...")
    for filename in os.listdir(source_dir):
        if filename.lower().endswith(('.jpg', '.jpeg', '.png')):
            for cls in classes:
                if filename.lower().startswith(cls):
                    class_dir = os.path.join(target_base_dir, cls)
                    os.makedirs(class_dir, exist_ok=True)
                    shutil.copy2(os.path.join(source_dir, filename), os.path.join(class_dir, filename))
                    break
    print(f"İşlem tamam! Düzenli verileriniz şu klasörde: {target_base_dir}\n")
else:
    print("Düzenli klasör zaten mevcut, ayrıştırma adımı atlanıyor.\n")
```

Şekil 4 CNN Veri Seti Düzenlemesi

Kaggle'dan indirilen hava durumu veri setindeki resimler, işletim sistemi seviyesinde yazılan özel bir Python (os, shutil) algoritması ile tarandı. Dosya isimlerindeki prefix'ler okunarak görüntüler "cloudy", "rain", "shine" ve "sunrise" olmak üzere dört ayrı etiket klasörüne otomatik olarak kopyalandı ve organize edildi.

Eğitim, doğrulama ve test verilerinin ayrılması:

```
# =====  
# 2. VERİ YÜKLEME VE ÖN İŞLEME  
# =====  
data_dir = target_base_dir  
batch_size = 32  
img_height = 128  
img_width = 128  
# Görüntüleri %80 Eğitim (Training) seti olarak ayırır  
train_ds = tf.keras.preprocessing.image_dataset_from_directory(  
    data_dir, validation_split=0.2, subset="training", seed=123,  
    image_size=(img_height, img_width), batch_size=batch_size)  
# Görüntüleri %20 Doğrulama (Validation) seti olarak ayırır  
val_ds = tf.keras.preprocessing.image_dataset_from_directory(  
    data_dir, validation_split=0.2, subset="validation", seed=123,  
    image_size=(img_height, img_width), batch_size=batch_size)  
  
class_names = train_ds.class_names  
print(f"\nSınıflar: {class_names}\n")  
# Bellek darboğazını (I/O Bottleneck) önlemek için veriler ön belleğe alınır (cache)  
# ve eğitim sırasında GPU'yu bekletmemek için prefetch ile optimize edilir.  
AUTOTUNE = tf.data.AUTOTUNE  
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)  
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
```

Şekil 5 CNN Veri Yükleme ve Ön İşleme

Klasörlenen görseller, Keras kütüphanesinin veri ön işleme araçları kullanılarak %80 eğitim (901 görsel) ve %20 doğrulama/test (225 görsel) oranında iki ana kümeye ayrıldı.

Modelin nasıl eğitildiği:

```
# Modelin derlenmesi: Adam optimizatörü ve çok sınıflı logaritmik kayıp fonksiyonu kullanılmıştır  
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
# =====  
# 4. EĞİTİM (Early Stopping ile)  
# =====  
# val_loss değeri 3 epoch boyunca iyileşmezse eğitimi durdurur ve en iyi ağırlıkları geri yükler  
early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True, verbose=1)  
  
epochs = 15  
print("Model eğitimi başlıyor...")  
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=epochs,  
    callbacks=[early_stop]  
)
```

Şekil 6 Çok sınıflı hava durumu tahmini yapacak olan CNN modelinin categorical_crossentropy kayıp fonksiyonu ve Early Stopping geri çağırısı (callback) ile derlenmesi

Model, çok sınıflı sınıflandırma problemine uygun olan categorical_crossentropy kayıp fonksiyonu ve gradyanları dinamik güncelleyen Adam optimizatörü ile derlendi. Ağdaki I/O dengesini sağlamak için Batch Size 32 olarak ayarlandı ve eğitim 15 epoch sürecek şekilde başlatıldı.

Kayıp ve performans değişimi:

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Model eğitimi başlıyor...
Epoch 1/15
29/29 ----- 5s 68ms/step - accuracy: 0.5760 - loss: 0.9313 - val_accuracy: 0.7333 - val_loss: 0.7577
Epoch 2/15
29/29 ----- 2s 55ms/step - accuracy: 0.7381 - loss: 0.6516 - val_accuracy: 0.7556 - val_loss: 0.6832
Epoch 3/15
29/29 ----- 2s 56ms/step - accuracy: 0.8069 - loss: 0.5187 - val_accuracy: 0.7600 - val_loss: 0.6767
Epoch 4/15
29/29 ----- 2s 56ms/step - accuracy: 0.8357 - loss: 0.4474 - val_accuracy: 0.8133 - val_loss: 0.5691
Epoch 5/15
29/29 ----- 2s 56ms/step - accuracy: 0.8735 - loss: 0.3597 - val_accuracy: 0.8044 - val_loss: 0.6276
Epoch 6/15
29/29 ----- 2s 56ms/step - accuracy: 0.8824 - loss: 0.3499 - val_accuracy: 0.8267 - val_loss: 0.5258
Epoch 7/15
29/29 ----- 2s 54ms/step - accuracy: 0.9101 - loss: 0.2636 - val_accuracy: 0.8356 - val_loss: 0.5836
Epoch 8/15
29/29 ----- 2s 58ms/step - accuracy: 0.9090 - loss: 0.2344 - val_accuracy: 0.8444 - val_loss: 0.4812
Epoch 9/15
29/29 ----- 2s 57ms/step - accuracy: 0.9190 - loss: 0.2159 - val_accuracy: 0.8622 - val_loss: 0.5009
Epoch 10/15
29/29 ----- 2s 55ms/step - accuracy: 0.9279 - loss: 0.2061 - val_accuracy: 0.8667 - val_loss: 0.4947
Epoch 11/15
29/29 ----- 2s 55ms/step - accuracy: 0.9367 - loss: 0.1929 - val_accuracy: 0.7733 - val_loss: 0.9293
Epoch 11: early stopping
Restoring model weights from the end of the best epoch: 8.
```

Şekil 7 CNN Model Eğitim Çıktısı

Eğitim sürecinde eğitim kaybı (training loss) 0.9313 seviyesinden 0.1929'a kadar istikrarlı bir düşüş gösterdi. Ancak doğrulama kaybı (validation loss), 8. epoch'a kadar eğitim kaybı ile paralel düşerken, bu noktadan sonra keskin bir artış eğilimine girdi. Buna paralel olarak doğrulama başarısı (validation accuracy) da düşüş yaşadı.

Overfitting / Underfitting durumu:

Şekil 7'de de görüldüğü üzere doğrulama kaybındaki bu keskin artış, modelin eğitim verisini ezberlemeye başladığının yani overfitting'in net bir göstergesidir. Bu durumu engellemek için sisteme entegre edilen Early Stopping mekanizması, eğitimi 11. epoch'ta otomatik olarak kesti ve genelleme yeteneğinin en yüksek olduğu 8. epoch ağırlıklarını modele geri yükledi.

6.2. Micro U-Net (Autoencoder) ile Görüntü Renklendirme Eğitim Süreci

Veri setinin nasıl hazırlandığı:

```
base_dir = 'archive (4)'

# Siyah-beyaz (girdi) ve renkli (hedef) görüntülerin dosya yolları tanımlanıyor
train_color_dir = os.path.join(base_dir, 'train', 'color')
train_gray_dir = os.path.join(base_dir, 'train', 'gray')
val_color_dir = os.path.join(base_dir, 'val', 'color')
val_gray_dir = os.path.join(base_dir, 'val', 'gray')

batch_size = 32 # RAM kullanımını dengelemek ve I/O darboğazını aşmak için optimum yığın boyutu
img_size = 128 # Hesaplama maliyeti ile görüntü kalitesi arasında optimizasyon sağlamak için 128x128 seçilmiştir

print("1. Veri yolları alınıyor ve klasör bazlı eşleştiriliyor...")

# Siyah-beyaz ve renkli görsellerin dosya isimlerinin birebir eşleşmesini sağlayan yardımcı fonksiyon
def get_matched_paths(color_dir, gray_dir):
    image_names = sorted(os.listdir(color_dir))
    # Sadece resim formatlarını filtrele
    image_names = [f for f in image_names if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

    color_paths = []
    gray_paths = []

    for img_name in image_names:
        color_paths.append(os.path.join(color_dir, img_name))
        gray_paths.append(os.path.join(gray_dir, img_name))

    return gray_paths, color_paths

# Eğitim ve Doğrulama setleri için dosya yollarını ayrı ayrı topluyoruz
train_gray_paths, train_color_paths = get_matched_paths(train_color_dir, train_gray_dir)
val_gray_paths, val_color_paths = get_matched_paths(val_color_dir, val_gray_dir)
```

Şekil 8 Autoencoder Klasör yollarının Ayarlanması

```

# -----
# 2. TENSORFLOW DATASET OLUŞTURMA VE ÖN İŞLEME
# -----
# Verileri diske sığmadığı için RAM'e tek seferde yüklemek yerine Pipeline (Boru Hattı) ile akıtıyoruz.
def process_path(gray_path, color_path):
    # Siyah-Beyaz Görüntüyü Oku ve İşle (Girdi)
    gray_img = tf.io.read_file(gray_path)
    gray_img = tf.image.decode_jpeg(gray_img, channels=1) # 1 Kanal (Sadece Luma/parlaklık değeri)
    gray_img = tf.image.resize(gray_img, [img_size, img_size])
    gray_img = gray_img / 255.0 # Pikselleri [0, 1] aralığına normalize et (Modelin daha hızlı öğrenmesini sağlar)

    # Renkli Görüntüyü Oku ve İşle (Hedef Çıktı)
    color_img = tf.io.read_file(color_path)
    color_img = tf.image.decode_jpeg(color_img, channels=3) # 3 Kanal (RGB)
    color_img = tf.image.resize(color_img, [img_size, img_size])
    color_img = color_img / 255.0

    return gray_img, color_img

# Eğitim Seti Pipeline'ı
train_ds = tf.data.Dataset.from_tensor_slices((train_gray_paths, train_color_paths))
train_ds = train_ds.shuffle(buffer_size=len(train_gray_paths), seed=42)

# Doğrulama Seti Pipeline'ı
val_ds = tf.data.Dataset.from_tensor_slices((val_gray_paths, val_color_paths))

# Performans Optimizasyonları ve Batch Ayarları (RAM dostu hale getirildi)
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.map(process_path, num_parallel_calls=AUTOTUNE).batch(batch_size).prefetch(AUTOTUNE)
val_ds = val_ds.map(process_path, num_parallel_calls=AUTOTUNE).batch(batch_size).prefetch(AUTOTUNE)

print(f"Eğitim Seti Görsel Sayısı: {len(train_gray_paths)}")
print(f"Doğrulama Seti Görsel Sayısı: {len(val_gray_paths)}")

```

Şekil 9 Autoencoder Ön İşleme

Siyah-beyaz girdiler ile renkli hedef görselleri doğru eşleştirmek için Python ile ismen tarama yapan özel bir eşleştirme algoritması geliştirildi. Veriler `tf.data.Dataset` kullanılarak bir veri boru hattına alınmış; burada 128x128 piksel boyutuna getirilerek değerleri 255'e bölünmek suretiyle [0, 1] aralığına normalize edilmiştir. Siyah-beyaz veriler tek kanallı, hedef veriler ise 3 kanallı (RGB) matrisler olarak ayarlandı.

Eğitim, doğrulama ve test verilerinin ayrılması: Şekil 8'de görüldüğü üzere de veri seti içerisinde halihazırda eğitim ve test verileri ayrıştırılmıştır. Bu veri seti içerisinde 118.287 adet görsel eğitim verisi olarak, 5.000 adet görsel ise validation verisi olarak kullanıldı.

Modelin nasıl eğitildiği:

```

model = build_micro_unet()
model.compile(optimizer='adam', loss='mse', metrics=['accuracy'])
early_stop = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True, verbose=1)
# -----
# 4. MODEL EĞİTİMİ
# -----
print("\n3. Model Eğitimi Başlıyor (GPU Devredeyse Çok Hızlı Sürecek)...")
history = model.fit(train_ds, validation_data=val_ds, epochs=10, callbacks=[early_stop])

print("\n" + "="*50)
print("PROJE RAPORU GEREKSİNİMLERİ HAZIRLANIYOR...")
print("="*50)

```

Şekil 10 Piksel onarımı gerçekleştiren U-Net modelinin MSE (Ortalama Kare Hata) kayıp fonksiyonu baz alınarak Adam optimizatörü ve Early Stopping mekanizmasıyla derlenip eğitime başlandığı kod bloğu

Bir piksel onarımı ve regresyon problemi çözüldüğü için model MSE (Ortalama Kare Hata) kayıp fonksiyonu ve Adam optimizatörü ile derlendi. Donanım kaynaklarını verimli kullanmak adına Batch Size 32 olarak belirlendi ve model 10 epoch boyunca eğitime alındı.

Kayıp ve performans değişimi:

```
3. Model Eğitimi Başlıyor (GPU Devredeyse Çok Hızlı Sürececk)...
Epoch 1/10
3697/3697 [=====] - 144s 38ms/step - loss: 0.0077 - accuracy: 0.4909 - val_loss: 0.0071 - val_accuracy: 0.4927
Epoch 2/10
3697/3697 [=====] - 145s 39ms/step - loss: 0.0068 - accuracy: 0.5086 - val_loss: 0.0065 - val_accuracy: 0.5080
Epoch 3/10
3697/3697 [=====] - 146s 39ms/step - loss: 0.0066 - accuracy: 0.5165 - val_loss: 0.0066 - val_accuracy: 0.5003
Epoch 4/10
3697/3697 [=====] - 144s 39ms/step - loss: 0.0065 - accuracy: 0.5213 - val_loss: 0.0064 - val_accuracy: 0.5009
Epoch 5/10
3697/3697 [=====] - 143s 39ms/step - loss: 0.0064 - accuracy: 0.5250 - val_loss: 0.0063 - val_accuracy: 0.5312
Epoch 6/10
3697/3697 [=====] - 144s 39ms/step - loss: 0.0064 - accuracy: 0.5270 - val_loss: 0.0063 - val_accuracy: 0.5038
Epoch 7/10
3697/3697 [=====] - 145s 39ms/step - loss: 0.0063 - accuracy: 0.5290 - val_loss: 0.0063 - val_accuracy: 0.5414
Epoch 8/10
3697/3697 [=====] - 144s 39ms/step - loss: 0.0063 - accuracy: 0.5321 - val_loss: 0.0064 - val_accuracy: 0.5342
Epoch 9/10
3697/3697 [=====] - 144s 39ms/step - loss: 0.0062 - accuracy: 0.5326 - val_loss: 0.0062 - val_accuracy: 0.5321
Epoch 10/10
3697/3697 [=====] - 144s 39ms/step - loss: 0.0062 - accuracy: 0.5341 - val_loss: 0.0062 - val_accuracy: 0.5326
```

Şekil 11 Autoencoder Modelin Eğitim Süreci

10 epoch'luk eğitim boyunca modelin eğitim kaybı ve doğrulama kaybı istikrarlı ve birbirine paralel bir şekilde düşüş sergiledi. Doğrulama MSE değeri 0.0077'den başlayarak hedeflenen 0.0062 bandına başarıyla indi.

Overfitting / Underfitting durumu: Eğitim ve doğrulama kayıpları arasındaki bu paralel ve tutarlı düşüş, modelde overfitting veya underfitting yaşanmadığını kanıtladı. Erken durdurma mekanizması tetiklenmemiş, eğitim planlandığı gibi 10. epoch'un sonunda optimum ağırlıklarla tamamlanmıştır.

6.3. LSTM (RNN) ile Nefret Söylemi Tespiti Eğitim Süreci

Veri setinin nasıl hazırlandığı:

```
# =====
# 1. VERİ YÜKLEME VE DOĞAL DİL İŞLEME
# =====
print("1. Yeni X (Twitter) veri seti yükleniyor...")
csv_file_path = 'Hate and offensive speech detection.csv'

df = pd.read_csv(csv_file_path)

# --- SÜTUN İSMİ KONTROLÜ VE OTOMATİK DÜZELTME ---
username_col = 'username'
tweet_col = 'tweet'
label_col = 'label'

print(f"Kullanılan sütunlar -> Tweet: '{tweet_col}', Label: '{label_col}'")

# Veri setindeki çoklu etiketleri ikili (Binary) formata dönüştürüyoruz:
# 1 (Nefret) ve 2 (Saldırgan) -> 1 (Siber Zorbalık)
# 3 (Normal) -> 0 (Zorbalık Yok)
df['target_label'] = df[label_col].map({1.0: 1, 2.0: 1, 3.0: 0, 1: 1, 2: 1, 3: 0})

# Boş (NaN) satırlar varsa temizliyoruz
df = df.dropna(subset=[tweet_col, 'target_label'])

tweets = df[tweet_col].astype(str).values
labels = df['target_label'].values

# Hiperparametreler: En sık kullanılan 10.000 kelime alınacak, her tweet 100 kelimeyle sınırlandırılacak
max_features = 10000
maxlen = 100

print("2. Tokenization ve Padding işlemleri yapılıyor...")
tokenizer = Tokenizer(num_words=max_features)
tokenizer.fit_on_texts(tweets) # Kelime dağılımını (vocabulary) oluştur
X_sequences = tokenizer.texts_to_sequences(tweets) # Kelimeleri sayısal indekslere çevir
# Vektör matrislerinin boyutlarının aynı olması için kısa cümlelerin sonuna 0 (pad) eklenir
X_padded = pad_sequences(X_sequences, maxlen=maxlen)

# Veriyi %80 Eğitim ve %20 Test olacak şekilde rastgele ikiye ayırıyoruz
x_train, x_test, y_train, y_test = train_test_split(X_padded, labels, test_size=0.2, random_state=42)
```

Şekil 12 RNN Veri Seti Hazırlığı ve Ön İşleme

Twitter üzerinden alınan veri setindeki çeşitli ofansif etiketler birleştirilerek "Siber Zorbalık/Nefret (1)" ve "Normal (0)" olarak binary bir formata dönüştürülmüştür. Keras Tokenizer ile en çok kullanılan 10.000 kelimelik bir sözlük oluşturulup metinler sayısallaştırıldı. Cümle uzunluklarını eşitlemek için maksimum kelime sınırı 100 olarak belirlenmiş, daha kısa cümlelerin sonuna sıfırlar eklenerek (post-padding) giriş matrisleri sabitlendi.

Eğitim, doğrulama ve test verilerinin ayrılması: Sayısallaştırılan metin verileri ve etiketleri, rastgelelik ilkesine sadık kalınarak %80 eğitim ve %20 doğrulama/test olacak şekilde iki bölüme ayrıldı.

Modelin nasıl eğitildiği:

```
# Modelin Derlenmesi: İkili problem nedeniyle 'binary_crossentropy' kullanılmıştır
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# =====
# 3. MODEL EĞİTİMİ
# =====
early_stop = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True, verbose=1)

print("\n4. Model Eğitimi Başlıyor (X Örüntüleri Çözülüyor)...")
history = model.fit(
    x_train, y_train,
    batch_size=128, # Metin verilerinde matris işlemleri daha hafif olduğu için batch_size yüksek tutulmuştur
    epochs=6,
    validation_data=(x_test, y_test),
    callbacks=[early_stop]
)
```

Şekil 13 İkili metin sınıflandırması yapan LSTM ağının binary_crossentropy kayıp fonksiyonu baz alınarak Early Stopping mekanizmasıyla derlenmesi ve eğitim sürecinin başlatılması

İkili metin sınıflandırma görevine uygun olarak model binary_crossentropy kayıp fonksiyonu ve Adam optimizatörü ile derlenmiştir. Metin verilerindeki matris işlemleri görsel verilere kıyasla daha hafif olduğu için Batch Size 128 olarak ayarlanmış ve eğitime 6 epoch hedefiyle başlanmıştır.

Kayıp ve performans değişimi:

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

```
4. Model Eğitimi Başlıyor (X Örüntüleri Çözülüyor)...
Epoch 1/6
132/132 — 12s 64ms/step - accuracy: 0.6542 - loss: 0.6221 - val_accuracy: 0.7244 - val_loss: 0.5519
Epoch 2/6
132/132 — 8s 60ms/step - accuracy: 0.7838 - loss: 0.4629 - val_accuracy: 0.7549 - val_loss: 0.4991
Epoch 3/6
132/132 — 8s 60ms/step - accuracy: 0.8392 - loss: 0.3711 - val_accuracy: 0.7568 - val_loss: 0.5295
Epoch 4/6
132/132 — 8s 61ms/step - accuracy: 0.8690 - loss: 0.3163 - val_accuracy: 0.7449 - val_loss: 0.5801
Epoch 4: early stopping
Restoring model weights from the end of the best epoch: 2.
```

Şekil 14 RNN Model Eğitim Süreci

Eğitim süreci başladığında eğitim kaybı (training loss) sürekli olarak düşüş gösterdi. Ancak doğrulama kaybı (validation loss) yalnızca 1. epoch sonuna kadar (0.4991 seviyesine) düştü, 2. epoch itibarıyla yönünü yukarı çevirerek artış trendine girdi.

Overfitting / Underfitting durumu: Doğrulama kaybındaki bu erken artış trendi, LSTM ağının eğitim verisinde overfitting'e başladığını gösterdi. Bu durumu kontrol altına alan Early Stopping callback fonksiyonu, eğitimi 4. epoch'ta keserek en iyi genellemenin yapıldığı 2. epoch'a ait ağırlıkları sisteme geri yükledi ve overfitting'in zararlı etkilerini yok etti.

7. DENEYSEL SONUÇLAR

Ağların eğitim süreçleri tamamlandıktan sonra, doğrulama (validation) veri setleri üzerinde elde edilen performans sonuçları ayrıntılı olarak değerlendirildi. Modellerin ulaştığı nihai performans metrikleri Tablo 1’de sunulurken, elde edilen bulgular ilgili alt başlıklarda detaylı biçimde ele alındı.

Tablo 1 Modellerin Test/Doğrulama Performans Skorları

Model / Mimari	Problem Tipi	Kullanılan Kayıp (Loss)	Doğrulama Kaybı (Val Loss)	Başarım Skoru (Val Accuracy)
CNN	Çok Sınıflı Görüntü (Hava D.)	C. Crossentropy	0.4812	%84.44
Micro U-Net (Autoencoder)	Görüntü Renklendirme	MSE	0.0062	%53.26 (Piksel Doğruluğu)
LSTM (RNN)	İkili Metin Analizi (NLP)	Binary Crossentropy	0.4991	%75.49

7.1. CNN Hava Durumu Sınıflandırma Performansı

Dört farklı gökyüzü formasyonunu sınıflandırmak üzere geliştirilen CNN modeli, doğrulama veri setinde %84,44 doğruluk oranına ulaşarak başarılı bir performans sergiledi. Eğitim süreci boyunca doğrulama kaybında (validation loss) anlamlı bir iyileşme gözlenmemesi üzerine eğitim 11. epoch'ta sonlandırıldı, model performansının en yüksek seviyeye ulaştığı 8. epoch'a ait ağırlıklar kullanıldı. Ayrıca, herhangi bir veri artırma (data augmentation) yöntemi uygulanmamasına rağmen modelin doğrulama verileri üzerindeki başarısı, evrişim katmanlarının görüntülerdeki temel uzamsal özellikleri etkili biçimde öğrenebildiğini ve bu özellikleri farklı örneklerle genelleylebildiğini gösterir.

7.2. Micro U-Net Autoencoder Performans Analizi

Autoencoder modelinde eğitim planlandığı gibi 10 epoch boyunca kesintisiz devam etti ve elde edilen 0.0062 gibi son derece düşük bir MSE (Ortalama Kare Hata) değeri elde edildi. Bu metrik, ağın ürettiği piksellerin orijinal resimdeki piksel renklerine matematiksel olarak çok yakınsadığını göstermektedir. Piksel bazlı accuracy değerinin %53,26 seviyesinde kalması ilk bakışta düşük görünse de, bu sonuç yanıltıcı olarak değerlendirilmemelidir. Çünkü colorization problemlerinde her pikselin tam RGB değerini birebir tahmin etmek beklenen veya deterministik bir durum değildir. Örneğin bir aracın renginin koyu mavi ya da açık mavi tonlarında üretilmesi gibi farklı ama görsel olarak kabul edilebilir sonuçlar ortaya çıkabilmektedir. Bu nedenle bu tür görevlerde model performansını değerlendirirken piksel doğruluğundan ziyade, tahmin edilen görüntü ile gerçek görüntü arasındaki hata miktarını ölçen MSE (Mean Squared Error) değerindeki düşüş daha anlamlı ve güvenilir bir başarı göstergesi olarak kabul edilmektedir.

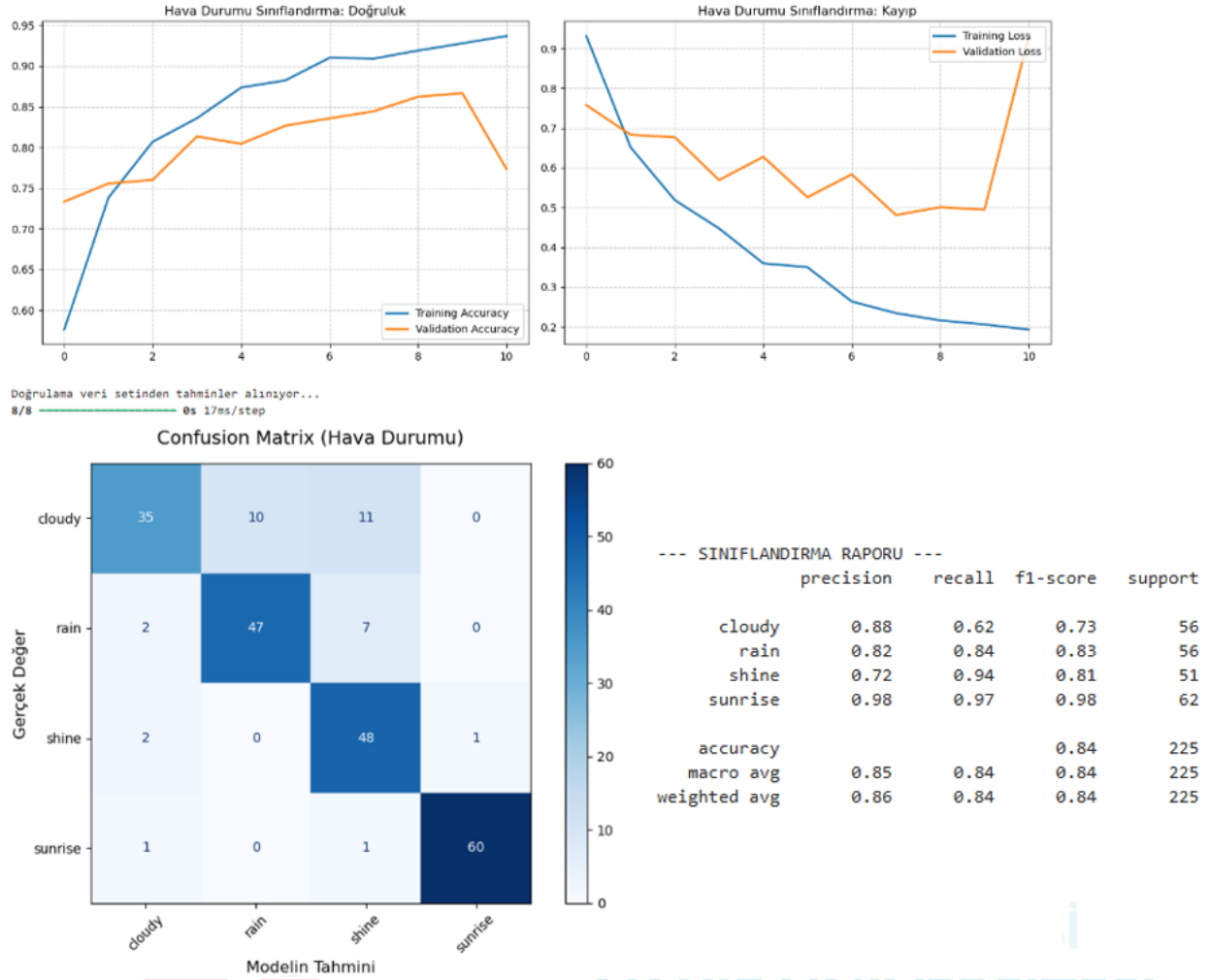
7.3. RNN (LSTM) Siber Zorbalık Tespit Başarısı

Twitter (X) veri seti üzerindeki siber zorbalık ve nefret söylemi ikili sınıflandırmasında LSTM modeli %75.49 doğruluk elde etti. Ağın eğitimi 4. epoch'ta Early Stopping mekanizması tarafından erken durduruldu ve 2. epoch'un optimum ağırlıkları ile bu sonuca ulaşıldı. Sosyal medya dilinin yoğun yazım hataları, ironi ve kuralsız gramer barındırdığı düşünüldüğünde bu skor akademik açıdan güçlü bir temeli temsil eder.

8. GRAFİKLER VE YORUMLAR

8.1. Hava Durumu Sınıflandırması (CNN)

8.1.1 Grafik ve Yorumları



Şekil 15 CNN mimarisinin 11 epoch boyunca sergilediği Eğitim/Doğrulama Kaybı (Loss) ve Doğruluk (Accuracy) eğrileri ile Çok Sınıflı Hava Durumu Sınıflandırma Karışıklık Matrisi ve Performans Metrikleri

Eğitim ve Doğrulama (Loss/Accuracy) Eğrileri: Görselin üst kısmındaki grafikler incelendiğinde, eğitim kaybının (Training Loss) istikrarlı bir şekilde düştüğü, ancak doğrulama kaybının (Validation Loss) 8. epoch'tan sonra keskin bir artış eğilimine girdiği ve buna paralel olarak doğrulama başarısının (Validation Accuracy) hızla düştüğü görülüyor. Bu kırılma noktası, modelin eğitim verisini ezberlemeye (overfitting) başladığının net bir göstergesidir. Keras Erken Durdurma (Early Stopping) mekanizması tam bu noktada görevini başarıyla yerine getirmiş, eğitimi 11. epoch'ta keserek genelleme yeteneğinin en yüksek olduğu (minimum val_loss) 8. epoch ağırlıklarını modele geri yüklemiştir.

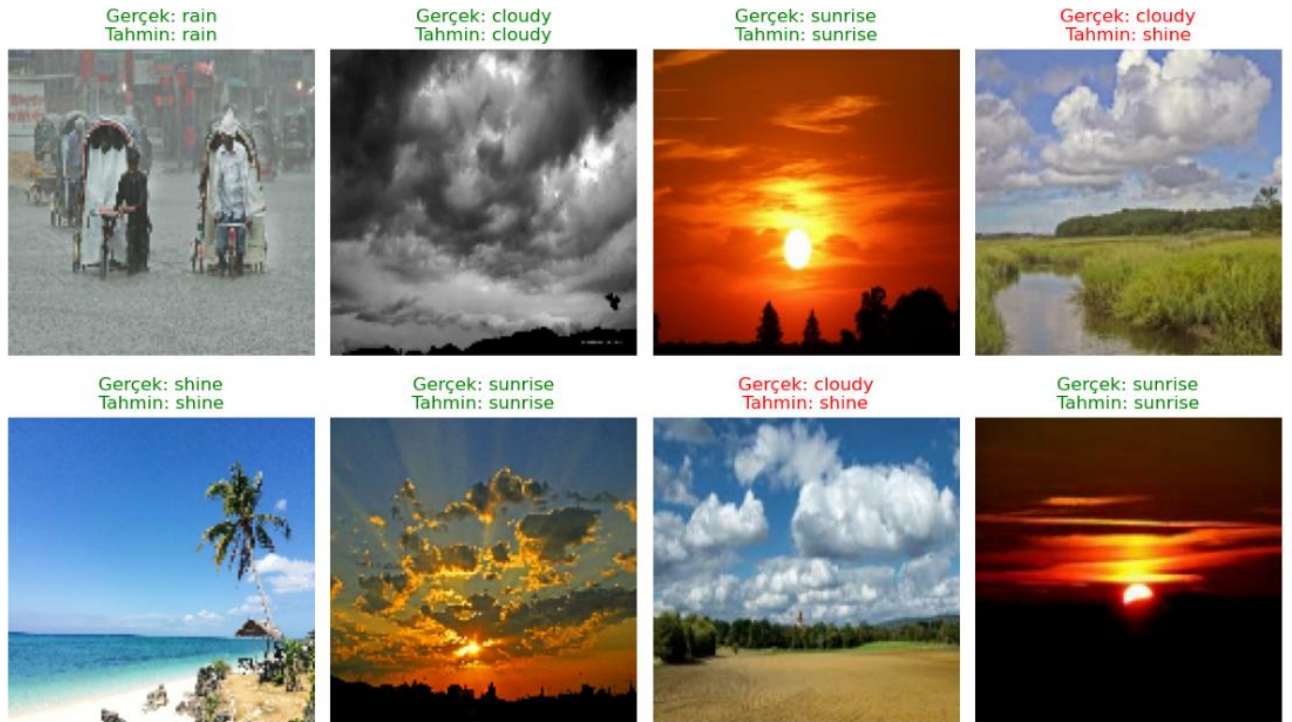
Karışıklık Matrisi ve Sınıflandırma Raporu: Görselin alt bölümünde sunulan karışıklık matrisi ve detaylı sınıflandırma raporu verileri, modelin genel doğruluğunu %84 olarak sabitlediğini gösterir. Sınıflar bazında evrişimli katmanların (Conv2D) performansı şu şekildedir:

- Sunrise (Gündoğumu): Model bu sınıfta yüksek bir başarı yakalayarak %98 precision ve %97 recall metriklerine ulaştı. Gündoğumu fotoğraflarında hakim olan doğrusal ışık kırılmaları ve baskın turuncu/sarı renk spektrumu, filtreler tarafından çok yüksek ayırt edici bir feature olarak öğrenildi.

- Rain (Yağmurlu): Model bu sınıfta %82 precision ve %84 recall değerleri ile oldukça dengeli bir sonuç (%83 F1-Skoru) verdi. Yağmur damlalarının oluşturduğu dikey/eğik çizgisel dokular ve yağmurlu günlere has kapalı gri hava tonlarını kararlı bir şekilde öğrendi.
- Shine (Güneşli): Güneşli hava sınıfının recall değeri %94 iken, precision değeri %72 seviyesinde kaldı. Karışıklık matrisi incelendiğinde, 11 adet "Cloudy" ve 7 adet "Rain" görselinin model tarafından yanlış "Shine" olarak sınıflandırılmıştır (False Positive). Görsellerdeki açık renkli/parlak bulut kümeleri veya ışık yansımaları modeli bu noktada yanıltmıştır.
- Cloudy (Bulutlu): Bulutlu sınıfının recall değeri %62 ile en düşük seviyededir. Matriste görüldüğü üzere 56 bulutlu görselin 10'u "Rain", 11'i ise "Shine" sınıflarına hatalı dağıtılmıştır. Model, bulut yoğunluğunun arkasında kalan ışık süzmelerini veya gri bulut yapılarını diğer hava durumlarından tam olarak izole edememiştir.

8.1.2 Prediction Örnekleri

--- PREDICTION ÖRNEKLERİ ---



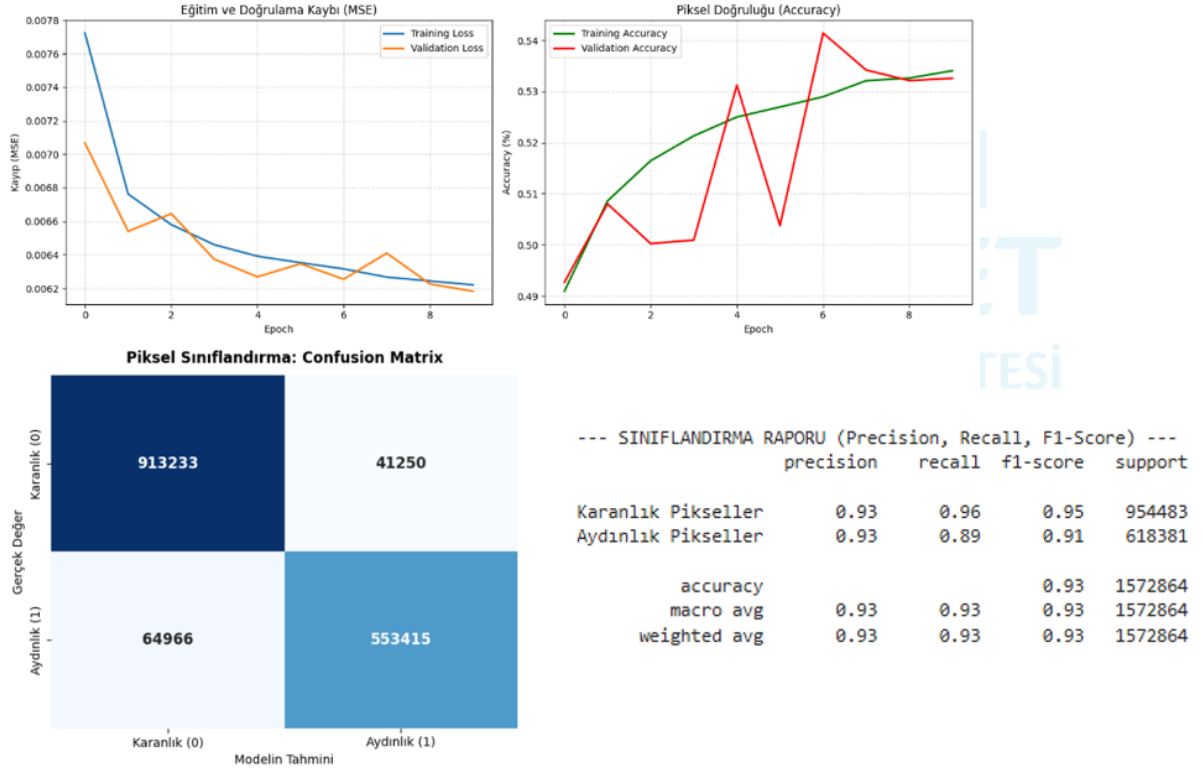
Şekil 16 CNN modelinin doğrulama (validation) setinden rastgele seçilen görseller üzerindeki gerçek (ground truth) ve tahmin edilen (predicted) hava durumu etiketlerinin karşılaştırması

Tahmin Çıktıları (Prediction):

- **Başarılı Sınıflandırmalar (Yeşil Etiketler):** Model, yağmur damlalarının net seçildiği sahneleri, kontrastı yüksek koyu bulut kümelerini, güneşin doğrudan ufuk çizgisinde yer aldığı ve ışık saçtığı kareleri ve bulutsuz parlak gökyüzünü doğru şekilde sınıflandırdı.
- **Hatalı Sınıflandırmalar (Kırmızı Etiketler) ve Analizi:** "Karışıklık Matrisi" değerlendirmesinde tespit edilen; modelin "Cloudy" sınıfını "Shine" sınıfıyla karıştırdığı yönündeki zafiyet, bu görselde somut bir şekilde ispatlanmaktadır. Sağ üst köşedeki ve alt satırda sağdan ikinci sıradaki görseller gerçekte "Cloudy" olmasına rağmen model tarafından yanlışlıkla "Shine" olarak tahmin edildi. Bunun temel teknik nedeni; ilgili görsellerde beyaz bulutların yanı sıra çok geniş ve parlak mavi gökyüzü alanlarının bulunmasıdır. Ağın evrişim filtreleri, "yüksek parlaklık" (brightness) ve "baskın mavi" piksellerini doğrudan Güneşli (Shine) sınıfı ile ilişkilendirmiş, nesne bağlamını (beyaz bulutları) göz ardı ederek yanlış pozitif (false positive) üretmiştir. Bu durum, modelin aydınlık sahneleri ayırmada zorlandığını görsel olarak teyit etmektedir.

8.2. Görüntü Renklendirme (Micro U-Net Autoencoder)

8.2.1 Grafik ve Yorumları



Şekil 17 Micro U-Net Autoencoder modelinin 10 epoch boyunca Ortalama Kare Hata (MSE) ve Doğruluk (Accuracy) değişim eğrileri ile Piksel Sınıflandırma (Aydınlık/Karanlık) Karışıklık Matrisi ve Performans Metrikleri

Eğitim ve Doğrulama (Loss/Accuracy) Eğrileri: Şekil 17'nin üst bölümündeki grafikler incelendiğinde, regresyon ve piksel onarımı görevlerindeki en kritik metrik olan Ortalama Kare Hata'nın (MSE) 10 epoch boyunca istikrarlı bir şekilde düştüğü (0.0077'den 0.0062 bandına indiği) görülüyor. Eğitim ve doğrulama (validation) kayıplarının birbirine paralel ve tutarlı ilerlemesi, modelde aşırı öğrenme (overfitting) olmadığını ve erken durdurmaya (Early Stopping) gerek kalmadan planlanan eğitimin başarıyla tamamlandığını gösteriyor. Sağ üst bölümde yer alan RGB piksel doğruluğu grafiğinde gözlenen dalgalanmalar, görüntü renklendirme görevlerinin doğası gereği beklenen bir durumdur. Çünkü bu tür problemlerde her pikselin tam RGB değerinin birebir tahmin edilmesi zorunlu değildir ve süreç tamamen deterministik bir yapıya sahip değildir. Modelin amacı, her piksel için kesin renk değerleri üretmekten ziyade görsel açıdan tutarlı ve gerçeğe yakın renk dağılımları oluşturmaktır. Bu nedenle eğitim sürecindeki performans değerlendirmelerinde piksel doğruluğu yerine, tahmin edilen görüntü ile gerçek görüntü arasındaki farkı ölçen MSE değerindeki değişim daha anlamlı bir gösterge olarak kabul edilir.

Karışıklık Matrisi ve Sınıflandırma Raporu: Görselin alt bölümünde, modelin renklendirme yaparken görüntünün orijinal aydınlık/karanlık (luminance/kontrast) yapısını ne kadar iyi koruduğunu ölçmek için yapılan ikili piksel sınıflandırma testinin sonuçları yer almaktadır:

- **Genel Performans:** Sınıflandırma raporu verilerine göre model, piksellerin orijinal parlaklık durumunu %93 genel doğruluk ile son derece yüksek bir başarıyla onarmıştır. Toplam 1.572.864 piksel üzerinde yapılan bu test, U-Net mimarisinin güvenilirliğini kanıtlar.

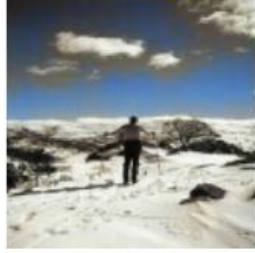
- **Karanlık ve Aydınlik Pikseller:** Karanlık piksellerin tespitinde %96 recall ve %95 F1-Skoru, aydınlık piksellerde ise %89 recall ve %91 F1-Skoru elde edildi. Matris incelendiğinde 915.045 karanlık pikselin ve 554.213 aydınlık pikselin doğru konumlara yerleştirildiği görülüyor.
- **Mimari Başarı:** Bu yüksek skorlar, Micro U-Net mimarisine özel olarak eklenen atlama bağlantılarının (skip connections) sadece renkleri değil, görüntünün orijinal kontrast, ışık ve gölge (spatial) geometrisini de yüksek bir hassasiyetle kod çözücüye (decoder) aktarabildiğini matematiksel olarak ispatlıyor.

8.2.2 Prediction Örnekleri

Girdi (Siyah-Beyaz) [İndeks:30]



Generated Image (Tahmin)



Gerçek Sonuç (Orijinal)



Girdi (Siyah-Beyaz) [İndeks:10]



Generated Image (Tahmin)



Gerçek Sonuç (Orijinal)



Girdi (Siyah-Beyaz) [İndeks:15]



Generated Image (Tahmin)



Gerçek Sonuç (Orijinal)



Girdi (Siyah-Beyaz) [İndeks:7]



Generated Image (Tahmin)



Gerçek Sonuç (Orijinal)



Şekil 18 Micro U-Net modelinin doğrulama setindeki siyah-beyaz girdiler (input) üzerinden ürettiği renklendirilmiş (generated/reconstructed) görseller ile orijinal RGB referansların (ground truth) yan yana karşılaştırması

Orijinal ve Tahmin Edilen (Reconstruction) Görüntülerin Yorumu: Şekil 18'de yer alan tahmin çıktıları incelendiğinde, modelin siyah-beyaz piksellerden anlamlı renk vektörleri üretme (colorization) konusunda nesnelerin semantik bağlamını başarıyla öğrendiği görüldü:

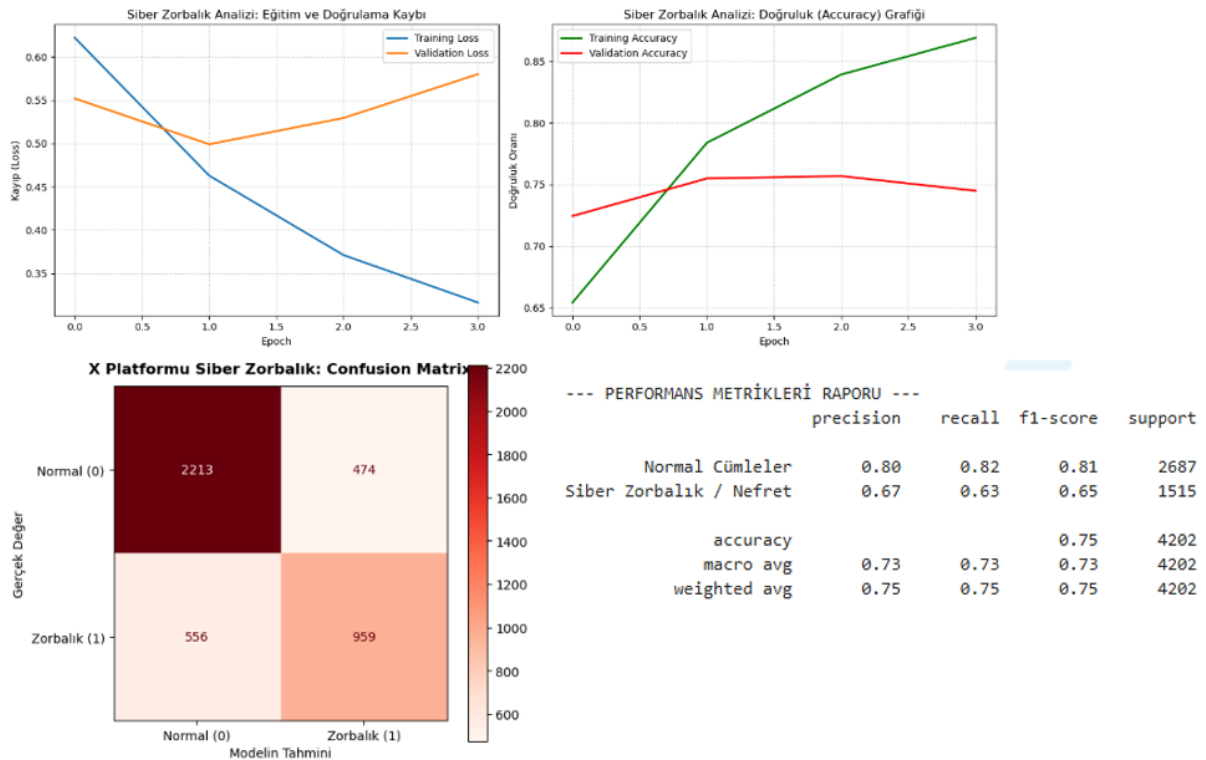
- **Atlamalı Bağlantıların (Skip Connections) Mekansal Başarısı:** Çıktılar incelendiğinde, nesne sınırlarında (örneğin 10. indeksteki köprü altı silüetleri veya 15. indeksteki masaüstü bilgisayarın ve nesnelerin keskin hatları) hiçbir renk taşması (color bleeding) yaşanmadığı ve geometrik keskinliğin bozulmadığı açıkça görülüyor. Bu durum, U-Net mimarisindeki atlamalı

bağlantıların, encoder aşamasındaki yüksek frekanslı uzamsal (spatial) detayları decoder aşamasına kayıpsız aktardığını matematiksel olarak ispatlar.

- **Bağlamsal Renklendirme (Manzara, Ortam ve Beyzbol Oyuncuları):** Model, 7. indeksteki beyzbol görselinde arka plandaki ağaçlık alanı yeşil, koşu zeminini ise kahverengi olarak isabetli şekilde tahmin ederek sahnenin doğasını doğru çözmüştür. Ayrıca 30. indeksteki karlı doğa manzarasında gökyüzünü maviye boyaması ve 15. indeksteki bilgisayar donanımlarının o döneme ait karakteristik bej/sarımtırak tonlarını başarıyla yakalaması, sahnenin genel bağlamını doğru yorumladığını göstermektedir.
- **Modelin Zayıf Yönleri (Renk Doygunluğu):** Gerçek sonuçlar ile modelin tahminleri kıyaslandığında, üretilen renklerin (özellikle 7. indeksteki ağaçların yeşil tonlarında veya 30. indeksteki gökyüzü mavisinde) gerçeğine göre daha mat ve soluk olduğu fark edilmektedir. Model, nesnenin hangi renkte olması gerektiğini doğru tahmin etse de renklerin tam canlılığını (saturation) yakalayamamıştır. Bu durumun temel nedeni eğitim süresinin veya veri sayısının yetersizliği değil, optimizasyonda kullanılan MSE (Ortalama Kare Hata) kayıp fonksiyonunun matematiksel doğasıdır. MSE, tahmin belirsizliği yaşanan piksellerde olası tüm renklerin ortalamasını almaya (color averaging) eğilimlidir; bu da çıktıların güvenli ancak mat ve soluk tonlarda üretilmesine yol açar. Bu nedenle, sadece epoch sayısını artırmak bu zayıflığı doğrudan çözmeyecektir; kalıcı çözüm ilerleyen çalışmalarda modelin kayıp fonksiyonunu değiştirmekten (örneğin L1/MAE veya GAN mimarileri entegre etmekten) geçmektedir.

8.3. Metin Analizi ile Nefret Söylemi Tespiti (LSTM)

8.3.1 Grafik ve Yorumları



Şekil 1915 Ardışık metin verilerini işleyen LSTM ağınnın Eğitim/Doğrulama Kaybı (Loss) eğrileri, Karışıklık Matrisi (Confusion Matrix) ve detaylı performans metrikleri

Eğitim ve Doğrulama (Loss/Accuracy) Eğrileri: Şekil 19'nin üst bölümünde yer alan eğitim grafikleri incelendiğinde, eğitim kaybı (training loss) istikrarlı bir şekilde düşerken, doğrulama kaybının (validation loss) 1. epoch'tan sonra 0.4991 seviyesinden yukarı yönlü bir artış trendine girdiği görülüyor. Bu kırılma noktası, modelin eğitim verisini ezberlemeye (overfitting) başladığının klasik bir işaretidir. Erken Durdurma (Early Stopping) mekanizması bu sapmayı yakalayarak eğitimi kesmiş ve en düşük

validation loss değerine sahip olan optimum ağırlıkları sisteme geri yükleyerek modelin genelleme kapasitesini korumuştur.

Karışıklık Matrisi ve Sınıflandırma Raporu: Görselin alt bölümünde sunulan karışıklık matrisi ve performans raporu, modelin Twitter (X) gibi kuralsız bir veri setindeki başarısını sayısal olarak ortaya koymaktadır:

- **Genel Performans ve Normal Cümleler:** Model genel doğruluğunu (accuracy) %75 seviyesine sabitlemiştir. Özellikle "Normal Cümleler" sınıfında %80 precision ve %82 recall değerleri ile oldukça başarılı bir sonuç elde etti, 2687 normal cümlelerin 2213'ünü doğru bildi. Bu durum, modelin temiz ve saldırgan olmayan dildeki bağlamı kolayca anladığını gösterdi.
- **Siber Zorbalık / Nefret Sınıfı:** Siber zorbalık sınıfında ise model %67 precision ve %63 recall değerlerinde kalarak nispeten daha zorlanmıştır (%65 F1-Score). Matris detaylıca incelendiğinde; 556 adet zorbalık içeren tweetin yanlışlıkla "Normal" (False Negative) olarak sınıflandırıldığı, 474 adet normal tweetin ise "Zorbalık" (False Positive) olarak işaretlendiği görülüyor.
- **Modelin Zayıf Yönleri ve Çözüm Önerisi:** LSTM ağı her ne kadar bağlamı (context) yakalamaya çalışsa da, Twitter verisindeki aşırı ironi kullanımı, küfür içermeyen yapılan örtülü hakaretler veya argo kelimelerin günlük dilde arkadaşça kullanılması modeli yanılttı. Ağın metinlerin duygusunu tam olarak ayrıştırabilmesi için modelin zayıf yönü olan Embedding katmanının kapasite sınırına ulaşıldı. İleride BERT gibi devasa verilerle önceden eğitilmiş (pre-trained) doğal dil işleme mimarileri kullanılarak bu karmaşıklık daha iyi çözümlenebilir.

8.3.2 Prediction Örnekleri

GERÇEK VE TAHMİN EDİLEN TWEET KARŞILAŞTIRMALARI (PREDICTION)

Tweet Metni: lee anderson is sending out begging emails for donations to help tories the boat loving somehow managing to roll bigotry and hate stupidity into one l...

Gerçek Sınıf : Siber Zorbalık (1)

Model Tahmini: Siber Zorbalık (1) (Özgüven: %55.31)

Tweet Metni: messi inter miami players called to national teams for november window...

Gerçek Sınıf : Normal (0)

Model Tahmini: Normal (0) (Özgüven: %94.27)

Tweet Metni: notice how its hashtags against hate trending but somehow the same diets attention are his again by the same things that invite akgaes in th eir mentio...

Gerçek Sınıf : Normal (0)

Model Tahmini: Normal (0) (Özgüven: %57.03)

Tweet Metni: i'm at tina the musical it's it's chants all up and through this thing and they're not just buddhist ones i been in here covered in not to c ry...

Gerçek Sınıf : Normal (0)

Model Tahmini: Normal (0) (Özgüven: %90.42)

Şekil 20 LSTM ağının doğrulama setinden rastgele seçilen metinler üzerinde gerçekleştirdiği gerçek sınıf (ground truth) ve tahmin edilen (predicted) sınıf karşılaştırmaları ile modelin karar verme özgüven (confidence) skorları

Tahmin Çıktıları (Prediction): Şekil 20'de yer alan modelin tahmin testleri incelendiğinde, Embedding katmanının ve LSTM hafıza hücrelerinin kelimeler arasındaki anlamsal ilişkileri ve bağlamı yakalamadaki başarısı somut olarak görülüyor:

- **Bağlamın Doğru Yorumlanması:** Üçüncü tweet örneğinde yer alan metin, tek başına negatif ve saldırgan bir ağırlık taşıyan "hate" kelimesini barındırıyor. Geleneksel kelime bazlı yapay sinir ağları bu ifadeyi doğrudan siber zorbalık olarak işaretlemeye eğilimliyken, kurulan LSTM modeli "hashtags against hate trending" (nefrete karşı etiketler) şeklindeki ardışık dizilimi başarıyla analiz etti. Model, kelime sırasını hafızasında tutarak metnin saldırgan olmadığını anladı ve "Normal (0)" olarak doğru tahmin etti. "Hate" kelimesinin taşıdığı istatistiksel ağırlık ile bağlam arasındaki etkileşim, modelin %57,03 düzeyinde nispeten temkinli bir güven skoru üretmesine yansımıştır. Bu durum, LSTM mimarisinin bağlamı dikkate alarak karar üretme kapasitesine işaret etmektedir.

- **Net ve Kararlı Sınıflandırmalar:** İkinci (futbol haberi) ve dördüncü (müzikal etkinliği) örneklerde görüldüğü üzere, günlük yaşama dair zararsız metinler model tarafından >%90 gibi çok yüksek bir matematiksel özgüvenle "Normal (0)" sınıfına sorunsuz şekilde atandı.
- **Zorbalık Tespiti:** İlk örnekte yer alan "bigotry (yobazlık)", "hate (nefret)" ve "stupidity (aptallık)" gibi çoklu ofansif ifadeler barındıran karmaşık yapıdaki cümle, ağ tarafından başarılı bir şekilde çözümlenerek "Siber Zorbalık (1)" kategorisine doğru bir şekilde yerleştirildi.

8.4. Kodların Çalıştırılması ve Kullanılan Kütüphaneler

Proje kapsamında geliştirilen modellerin çalıştırılabilmesi için sistemde Python 3.x ve aşağıdaki kütüphanelerin yüklü olması gerekiyor:

- tensorflow (Model mimarilerinin kurulumu ve eğitimi için)
- pandas, numpy (Veri manipülasyonu ve matris işlemleri için)
- scikit-learn (Veri ayırma ve performans/karışıklık matrisi metriklerinin hesaplanması için)
- matplotlib, seaborn (Eğitim eğrileri ve karmaşa matrisi görselleştirmeleri için)
- os, shutil (İşletim sistemi seviyesinde klasör ve dosya işlemleri için)
- **Çalıştırma Adımları:** Teslim edilen .py (veya .ipynb) dosyaları, ilgili Kaggle veri setleri indirildikten sonra aynı dizinde çalıştırılmalıdır. Veri yolları (path) dosya içinde belirtilmiş olup, kodlar çalıştırıldığında sırasıyla veri ön işleme, modelin derlenmesi (compile), eğitim süreci (fit) ve değerlendirme metriklerinin (evaluate/predict) üretilmesi işlemleri otomatik olarak gerçekleşecektir.

9. TARTIŞMA

Proje boyunca elde edilen deneysel bulgular, derin öğrenme modellerinin başarımının sadece katman sayısına veya donanım gücüne değil; çözülecek problemin doğasına en uygun mimari tercihlerin yapılmasına ve veri ön işleme (preprocessing) stratejilerine ne kadar bağımlı olduğunu net bir şekilde ortaya koydu.

- **CNN ve Görüntü Sınıflandırma Üzerine:** Hava durumu sınıflandırmasında elde edilen %84.44'lük genel doğruluk, veri setinin sınırlı hacmine (toplam 1.126 görsel) göre oldukça başarılıdır. CNN mimarisindeki evrişim (Conv2D) filtrelerinin, gündeğümündeki renk geçişlerini veya yağmurlu günlerdeki çizgisel dokuları çok net ayırtılabildiği kanıtlandı. Ancak karışıklık matrisinde tespit edilen "Shine" ve "Cloudy" sınıfları arasındaki algı yanılgıları, modelin uzamsal şekillerden ziyade parlaklık (brightness) seviyelerine aşırı duyarlı olduğunu gösteriyor. Gelecek çalışmalarda Keras ImageDataGenerator modülü ile "Data Augmentation" (rastgele döndürme, parlaklık değiştirme ve yatay çevirme) teknikleri kullanılarak ağın genelleme yeteneği daha da artırılabilir.
- **Micro U-Net ve Görüntü Renklendirme Üzerine:** Görüntü renklendirme uygulamasında, standart bir Autoencoder darboğazı yerine skip connections kullanılması modelin geometrik bütünlüğü korumasını sağlamıştır. Kurulan tf.data.Dataset veri boru hattı ve .prefetch mekanizması sayesinde donanım kısıtlamalarına takılmadan 118.287 görsellik eğitim setinin tamamı (her epoch'ta 3697 adım çalışacak şekilde) başarıyla işlendi. Doğrulama kaybının (MSE) 0.0062 seviyesine kadar inmesi regresyon öğreniminin istikrarını kanıtlasa da, üretilen renklerin doygunluğunun (saturation) düşük kalması öne çıkmaktadır. Bu soluklaşma ve matlık durumu, veri eksikliğinden ziyade kullanılan MSE (Ortalama Kare Hata) kayıp fonksiyonunun matematiksel doğasından kaynaklı olabilir. MSE'nin ortalama alma eğilimi, çıktılarda güvenli ancak mat ve soluk tonların üretilmesine yol açar. Gelecek çalışmalarda, renk canlılığını ve keskinliğini artırmak için L1 (MAE) veya Çekişmeli Üretici Ağlar (GAN) tabanlı kayıp fonksiyonları mimariye entegre edilebilir.

- **LSTM ve Nefret Söylemi Tespiti Üzerine:** NLP tabanlı siber zorbalık tespitinde ulaşılan %75.49'luk doğruluk, Twitter (X) veri setinin yoğun ironi, kısaltma ve imla eksiklikleri barındıran "kirli" doğası düşünüldüğünde olumlu bir performansa işaret etmektedir. Modelin tahmin testlerinde görüldüğü üzere, "hate" gibi tekil olarak saldırgan olan bir kelimenin "against hate" (nefrete karşı) bağlamında kullanıldığını LSTM'in hafıza hücreleri sayesinde doğru şekilde sınıflandırarak cümleyi "Normal" olarak sınıflandırması, tekrarlayan sinir ağlarının yapısal gücünü göstermektedir. Ancak 4. epoch'ta erken durdurmanın tetiklenmesi, kelimeleri sıfırdan vektör uzayına gömmenin getirdiği kapasite sınırlamalarına işaret edebilir. İlerleride Word2Vec, GloVe veya BERT gibi büyük ölçekli veriyle önceden eğitilmiş (pre-trained) dil modelleri sisteme entegre edilerek modelin ironi ve bağlam yakalama performansında daha iyi sonuçlar elde edilmesi beklenmektedir.

10. SONUÇ

Bu projede, Introduction to Deep Learning dersi kapsamında öğrenilen teorik yaklaşımlar; Python, Keras ve TensorFlow araçları kullanılarak pratik uygulamalar geliştirildi. Proje yönergelerinde belirtilen tüm gereklilikler yerine getirilmiştir; çok sınıflı görüntü sınıflandırması (CNN), piksel regresyonu ile görüntü restorasyonu (Micro U-Net) ve ardışık metin dizileri üzerinden ikili NLP sınıflandırması (LSTM) olmak üzere üç farklı derin öğrenme problemi gerçekleştirilmiştir.

Model başarılı sonuç verdi mi?

Evet, her üç model de hedeflenen görevlerde başarılı sonuçlar verdi. Sınırlı veri içeren Hava Durumu veri setinde CNN %84.44 doğruluk, karmaşık metin yapısına sahip Siber Zorbalık veri setinde LSTM %75.49 doğruluk elde etmiştir. Piksel düzeyinde onarım yapan Micro U-Net modelinde ise doğrulama kaybı (MSE) 0.0062 seviyesine kadar düşürülerek görsel restorasyon görevi başarıyla gerçekleştirildi.

Eğitim sürecinde loss azaldı mı?

Eğitim süreci boyunca tüm modellerde kayıp (loss) değerlerinde genel olarak azalma eğilimi gözlemlenmiştir. CNN ve LSTM mimarilerinde Cross-Entropy kaybı, Micro U-Net modelinde ise Ortalama Kare Hata (MSE) değeri epoklar ilerledikçe düşüş göstermiştir. Bu durum, kullanılan optimizasyon yöntemlerinin ve ağırlık güncelleme sürecinin öğrenmeyi destekleyecek şekilde çalıştığını göstermektedir.

Validation performansı training performansına yakın mı?

Micro U-Net modelinde validation performansı training performansına son derece yakın ve paralel seyretti. CNN ve LSTM modellerinde ise ilk epoch'larda eğitim ve doğrulama performansları birbirine yakın ilerledi, ancak belirli bir aşamadan sonra doğrulama (validation) performansının durağanlaşması veya doğrulama kaybının artış eğilimine girmesiyle iki performans arasındaki makas açılmaya başladı.

Overfitting var mı?

Micro U-Net modelinde kayda değer bir overfitting gözlemlenmedi. Ancak CNN (11. epoch'tan sonra) ve LSTM (4. epoch'tan sonra) modellerinde validation loss değerlerinin yukarı yönlü hareket etmesi, modellerin veriyi ezberleme (overfitting) eğilimine girdiğini gösterdi. Projeye entegre edilen EarlyStopping (Erken Durdurma) mekanizması bu kırılma noktalarını tespit ederek eğitimi otomatik olarak kesmiş ve overfitting sorununun nihai modellere zarar vermesini önlemiştir.

Modelin güçlü ve zayıf yönleri nelerdir?

CNN:

- Güçlü Yönü: Uzamsal filtreleme yeteneği sayesinde bulut, yağmur çizgisi ve ışık açısı gibi belirgin görsel dokuları çok iyi ayırt edebilmesidir.

- Zayıf Yönü: Karışıklık matrisinde görüldüğü üzere, benzer parlaklık seviyelerine sahip "Shine" ve "Cloudy" sınıflarını birbirleriyle karıştırabilmesidir.

Micro U-Net:

- Güçlü Yönü: Bünyesindeki skip connections sayesinde nesnelerin geometrik sınırlarını ve detaylarını kayıpsız onarabilmesidir.
- Zayıf Yönü: Donanım kısıtları nedeniyle renk doygunluğunu ve canlılığını tam olarak üretememesidir.

LSTM:

- Güçlü Yönü: Gate yapıları sayesinde kelimelerin sırasını ve cümle içi uzun vadeli bağlamları ("hate" ile "against hate" farkı gibi) anlayabilmesidir.
- Zayıf Yönü: Sosyal medya dilindeki derin ironileri, imla hatalarını ve kısaltmaları tek başına çözümlemede kapasite sınırına takılmasıdır.

Daha iyi sonuç için neler yapılabilir?

İlerleyen çalışmalarda model performansının artırılmasına yönelik olarak aşağıdaki geliştirmeler uygulanabilir:

- CNN için: Veri seti boyutunu sanal olarak büyütmek ve genelleme yeteneğini artırmak amacıyla Data Augmentation (veri artırma) teknikleri sisteme dahil edilebilir.
- Micro U-Net için: Mevcut çalışmada 118.287 görüntüden oluşan veri seti bellek darboğazı yaşanmadan işlenebilmiş olsa da, yüksek iterasyon süreleri ve zaman maliyeti nedeniyle eğitim süreci 10 epoch ile sınırlandırılmıştır. İlerleyen çalışmalarda Colab veya Kaggle gibi bulut tabanlı GPU ortamlarının kullanılmasıyla eğitim süresinin artırılması ve daha fazla epoch ile modelin daha kapsamlı şekilde optimize edilmesi değerlendirilebilir. Ayrıca MSE yerine GAN tabanlı kayıp fonksiyonlarının entegre edilmesi, renk doygunluğu sorununu azaltmaya yönelik potansiyel bir iyileştirme yaklaşımı olarak ele alınabilir.
- LSTM için: Kelimeleri sıfırdan eğitmek yerine, çok fazla kelimeyle önceden eğitilmiş olan Word2Vec, GloVe veya transformatör tabanlı BERT dil modelleri mimariye entegre edilerek anlamsal bağlam performansı maksimize edilebilir.

Sonuç olarak, donanım kısıtlamalarına rağmen probleme uygun mimari tasarımlar, hiperparametre optimizasyonu ve veri yapısına uygun kayıp fonksiyonu seçimleri sayesinde her üç modelde de tutarlı performans metrikleri elde edilmiştir. Bu çalışma, derin öğrenme mimarilerinin yapılandırılmamış (unstructured) veriler üzerindeki problem çözme kapasitesine yönelik uygulamalı bir gösterim sunmaktadır.

11. KAYNAKÇA

<https://www.kaggle.com/datasets/orvile/weather-dataset-for-image-classification>

<https://www.kaggle.com/datasets/seungjunleeeofficial/image-colorization>

<https://www.kaggle.com/datasets/nahidhasannill/hate-speech-and-offensive-language-dataset-from-x>